

Curso 2025-26

# L3-1: Unity3D Introducción y Scripting

Fundamentos de Videojuegos

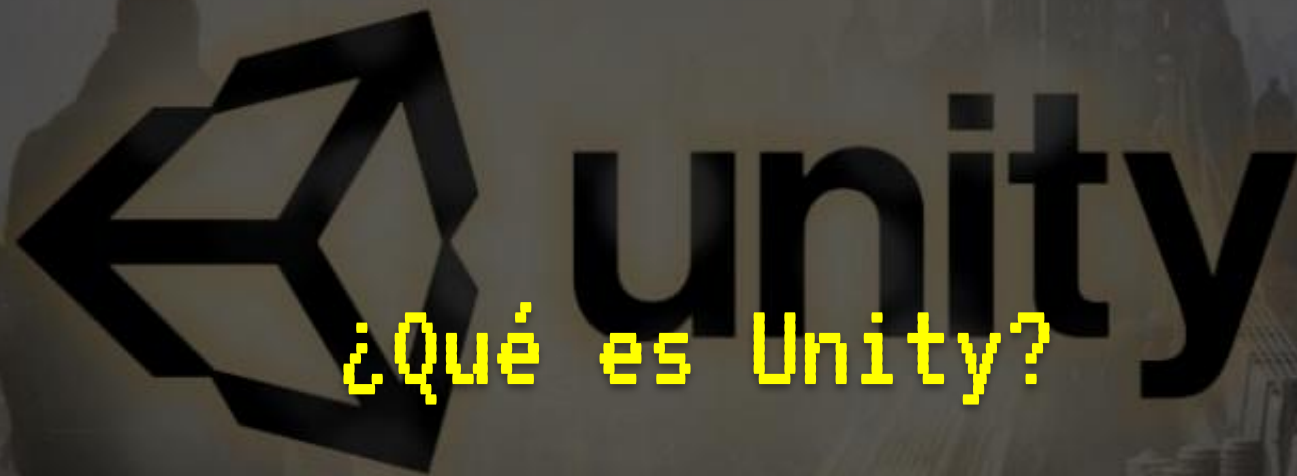
Jose María Barambones

DLSIS - ETSI Informáticos

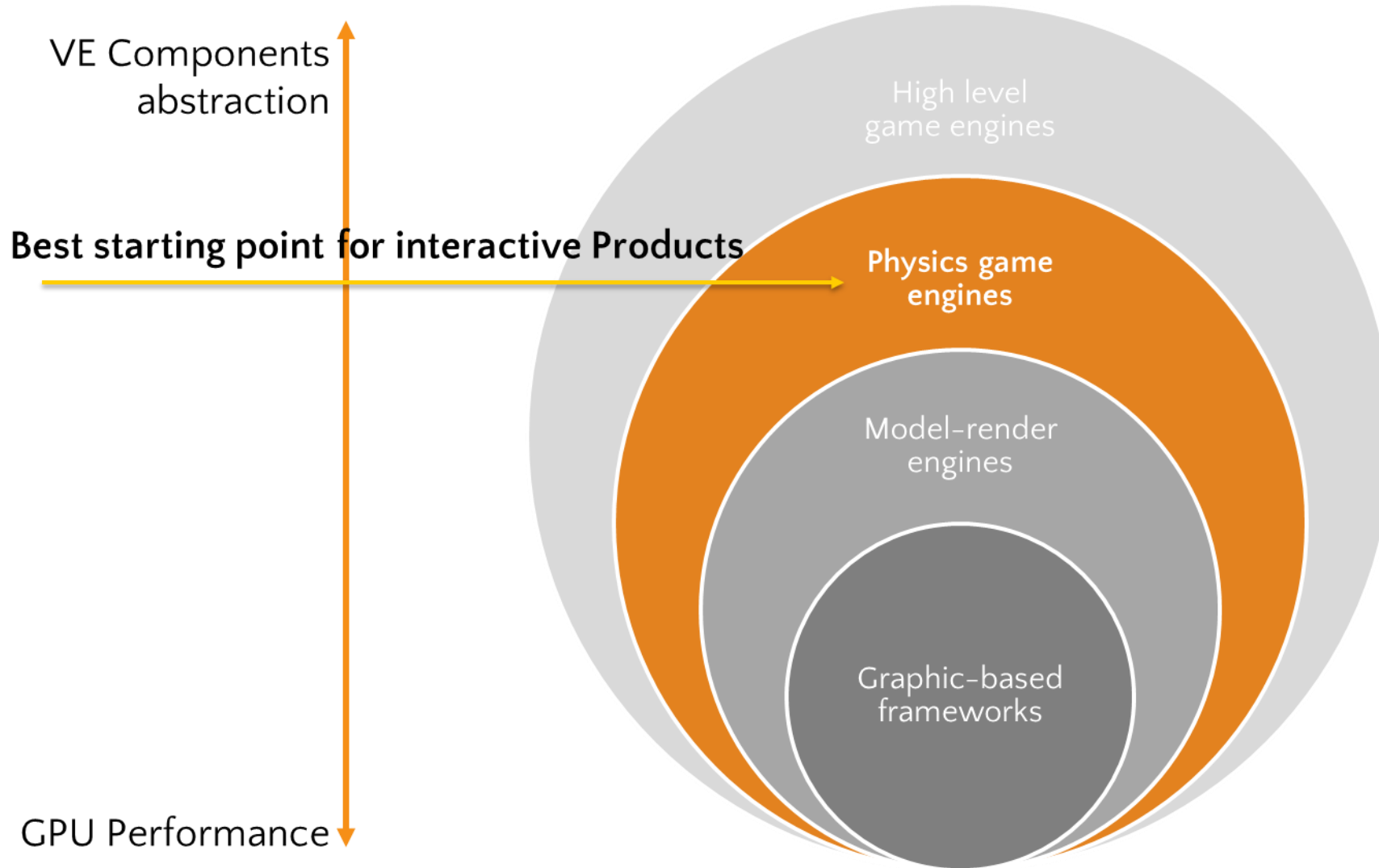
Universidad Politécnica de Madrid

# L3-1: Unity3D - Introducción y Scripting

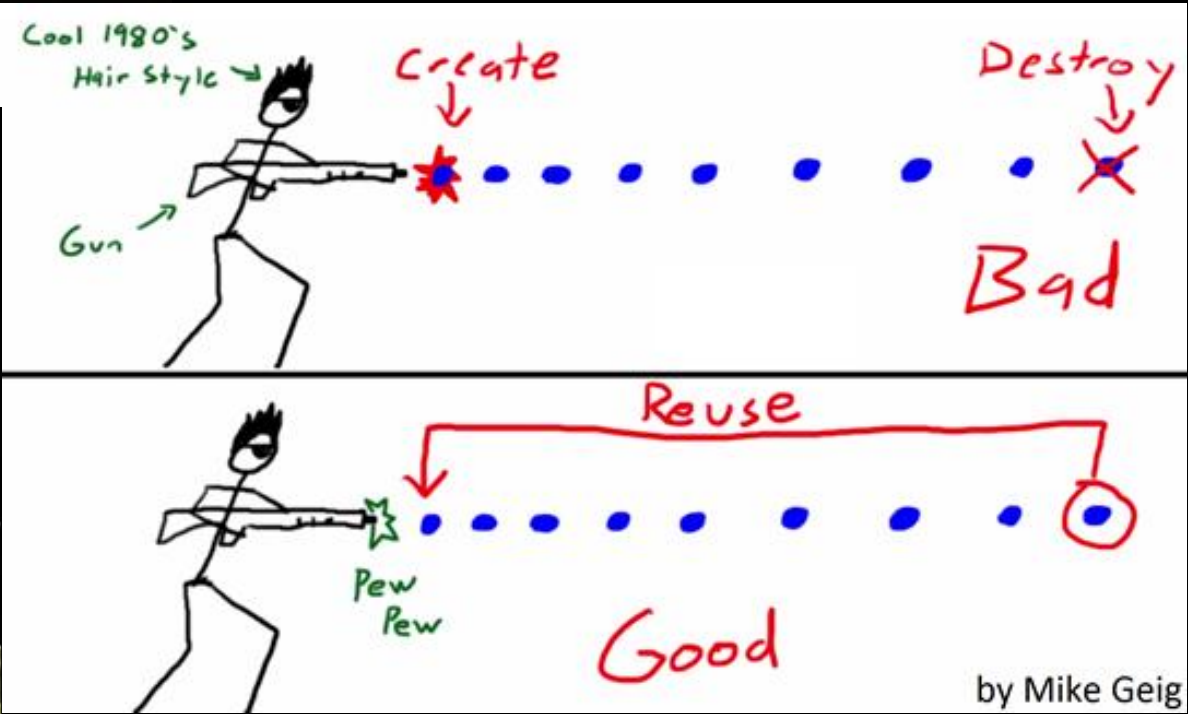
- ¿Qué es Unity?
- C# Scripting
  - Data types
  - Operadores
  - Value types
  - Reference types
  - C# OOP
  - Estructuras de datos
- Introducción a Unity
  - Configuración proyecto y repositorio
  - Cámara y Escena
  - GameObjects y Prefabs
  - Componentes
    - Transform
    - Collider
    - Rigidbody
    - Script
  - MonoBehaviour
  - Scripting con GameObjects



# Un poco de contexto



# Un poco de contexto





# ¿Qué es Unity?

- Unity es un motor de videojuegos multi-plataforma basados en físicas y objetos (prefabs).
- Proyectos 2D:
  - Gráficos basados en Sprites.
  - Imágenes planas con una cámara sin perspectiva.
- Proyectos 3D:
  - Utilizan modelos 3D con materiales (texturas) en sus superficies.
  - Cámara en perspectiva u ortográfica.
  - En esta categoría también entran los llamados 2.5D.
- En 2018, la mitad de los videojuegos para móvil y el 60% de los juegos en RV/RA fueron desarrollados en Unity.
- Unity fuera del videojuego: Cine, Automoción, Arquitectura, Ingeniería...

# ¿Qué es Unity?



# ¿Qué es Unity?

## GameObjects

- Un GameObject se compone de al menos un componente *Transform* que contiene la información sobre:
  - Posición
  - Escala
  - Rotación.
- El inspector lista todos los componentes del objeto enfocado.

## Prefabs

- GameObjects reutilizables una vez configurados a nuestro gusto en una escena.
- Los cambios a un prebab se propagan a todas sus instancias.
- Los cambios a un objeto de un prefab pueden mantener sus cambios de forma individual.

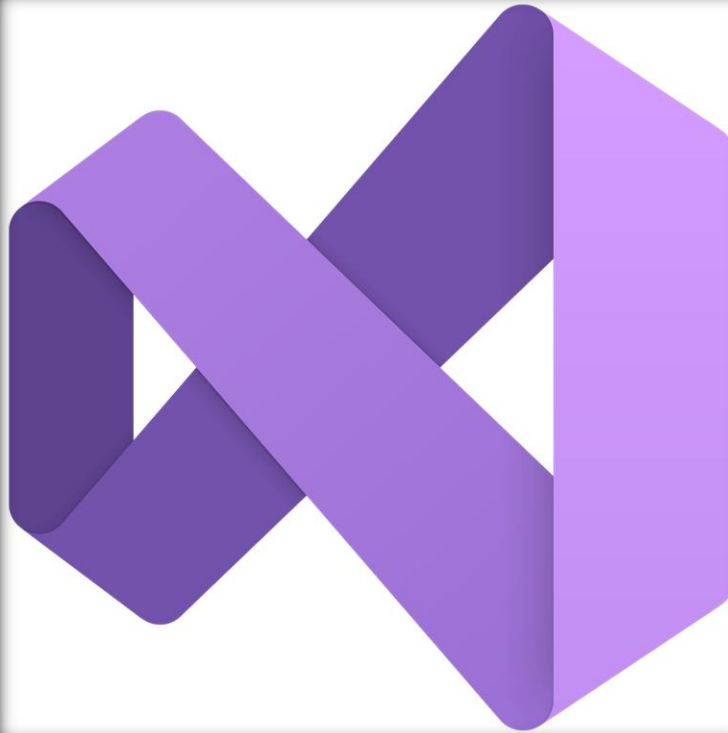


# Herramientas de desarrollo

Visual Code



Visual Studio



Rider



WebApplication > C# Program.cs

```
1  var builder = WebApplication.CreateBuilder(args);
2  var app = builder.Build();
3
4  app.MapGet("/", () => "Hello World!");
5
6  app.Run();
7
```

C# Scripting

# C# Hello World!

```
using System;
```

```
// namespace declaration
```

```
namespace HelloWorldApp {
```

```
    class Geeks {
```

```
        static void Main(string[] args) {
```

```
            // printing Hello World!
```

```
            Console.WriteLine("Hello World!");
```

```
            // To prevents the screen from closing quickly
```

```
            Console.ReadKey();
```

```
        }
```

```
    }
```

```
}
```

---

```
> /mnt/c/Users/Txema> csc hello.cs
```

```
> /mnt/c/Users/Txema> ./hello
```

# C# Data types

- **Value Types**

- Primitivos: `int, float, double, bool, char`
- Structs: `Vector3, Quaternion`

- **Reference Types**

- Objetos: `Player`
- Interfaces: `MonoBehaviour`
- Delegados: `delegate void MyFirstDelegate(int param1, int param2);`

<https://learn.unity.com/tutorial/tipos-de-datos>

# C# Operadores

- Aritméticos: + - \* / % ++ --
- Relacionales: == != < > >= <=
- Lógicos: && || !
- Bitwise: & | ^ << >>
- Asignación: = += -= \*= /= %= <<= >>= &= ^= |=
- Misceláneo: `sizeof(type)` `typeof(type)` & \* is as

- Prioridad algebraica:

*Postfix > Unary > Multiplicative > Additive > Relational > Equality > Logical > Assignment*



# C# Value Types: Enums y Structs

- Enumerados

- Tipo de datos donde el valor se toma dentro de un conjunto finito.

```
enum Dia { Lun, Mar, Mie, Jue, Vie, Sab, Dom };
```

- Por defecto, los valores están internamente asociados a un valor entero, empezando desde 0. Los valores o el tipo pueden especificarse.

```
enum Dia : ulong { Lun=1, Mar=5, Mie, Jue, Vie, Sab, Dom };
```

- Structs

- Tipo de datos para agrupar variables:

```
public struct Coords
```

```
{
```

```
    public float x;
```

```
    public float y;
```

```
}
```

- Pueden utilizarse, por ejemplo, para definir variables que representen una posición en un espacio de dos dimensiones

```
Coords p1 = new Coords();
```

```
p1.x = 0;
```

```
p1.y = 5;
```

# C# Reference Types

- Los datos como referencia contienen la dirección en memoria del dato.

```
Numero a = new Numero(1);  
Numero b = a;  
Debug.Log($"A: {a.valor}, B: {b.valor}"); // A: 1, B: 1  
a.valor = 5;  
Debug.Log($"A: {a.valor}, B: {b.valor}"); // A: 5, B: 5
```

- Un dato por referencia puede apuntar a “nada”: **null**

```
Persona ana = new Persona("Ana", 0);  
Persona juan = null;  
int edadAna = ana.edad; // 0  
int edadJuan = juan.edad; // error NullReferenceException
```

# C# Reference Types

- C# es un lenguaje **fuertemente tipado**.
- Las variables declaradas en un ámbito local pueden declararse de forma **implícita**:

```
var i = 10;           // Implicitly typed.
```

```
int i = 10;           // Explicitly typed.
```

- C# Permite hacer **conversiones implícitas** para tipos numéricos siempre y cuando no vayamos a perder información al pasar de un tipo a otro.

```
float a = 0;           // Ahora es un int
```

- **Casting**

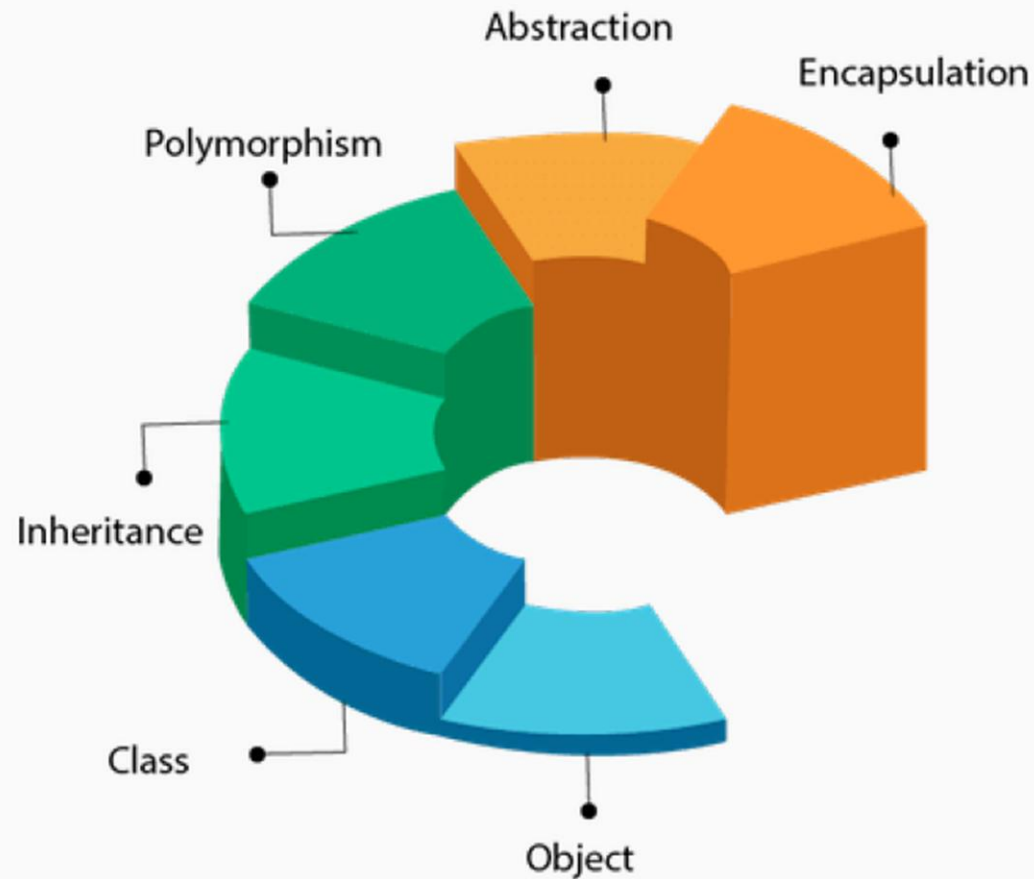
```
int a = (int)15.5;      // Ahora es un int sin decimales
```

- Las operaciones devuelven el tipo primitivo de **mayor precisión**:

```
var result1 = 5/2;      // Sigue siendo int
```

```
var result2 = 0.5f/2;   // Ahora es un float
```

## OOPs (Object-Oriented Programming System)



# C# OOP: Herencia

```
class Persona {  
    public string nombre;  
    public int edad;  
    public Persona(string nombre, int edad) {  
        this.nombre = nombre;  
        this.edad = edad;  
    }  
}  
  
class Trabajador : Persona {  
    public string puesto;  
    public Trabajador(  
        string nombre, int edad, string puesto) : base(nombre, edad) {  
        this.puesto = puesto;  
    }  
}
```

# C# OOP: Sobreescritura

```
class Persona {  
    public string nombre;  
    public int edad;  
    public virtual string Saludar() {  
        return "Hola, soy " + nombre;  
    }  
}  
  
class Trabajador : Persona {  
    public string puesto;  
    public override string Saludar() {  
        return base.Saludar() + ", y mi puesto es " + puesto;  
    }  
}
```

# C# OOP: Abstracción

```
public abstract class Employee {  
    public string Name { get; set; }  
    public double Salary { get; set; }  
    public abstract void IncreaseSalary(double rate);  
}  
  
public class SalesRepresentative : Employee {  
    public double AnnualSalesAmount { get; set; }  
    public override void IncreaseSalary(double rate) {  
        if (this.AnnualSalesAmount > 100000) {  
            this.Salary += (this.Salary * (rate + 5) / 100);  
        } else {  
            this.Salary += (this.Salary * rate / 100);  
        }  
    }  
}
```



# C# OOP: Interfaces

```
interface ISaludador { string Saludar(); }
```

```
class Entrenador : ISaludador {  
    public string nombre;  
    public virtual string Saludar() {  
        return "Soy " + this.nombre + " y te reto a un combate!";  
    }  
}
```

```
class Pokemon : ISaludador {  
    public string nombre;  
    public override string Saludar() {  
        return this.nombre;  
    }  
}
```

# C# Estructuras de datos: Arrays

- Unidimensionales

```
string[] cars = {"Volvo", "BMW", "Ford", "Mazda"};  
for (int i = 0; i < cars.Length; i++)  
    Console.WriteLine(cars[i]);
```

- Multidimensionales

```
var palabras = new string[3, 5]; // Matriz de 3x5 strings  
string[,] masPalabras = { {"hola", "hello"},  
                           {"adiós", "bye"} }; // 2x2  
var hola = masPalabras[0,0]; // "hola"
```

- Escalonadas ("Jagged")

```
int[][] jaggedArray = new int[][] {  
    new int[] { 1, 3, 5, 7, 9 },  
    new int[] { 0, 2, 4, 6 },  
    new int[] { 11, 22 }  
};
```

# C# Estructuras de datos: Listas

- Estructura de datos dinámica (*System.Collections.Generic*). Los elementos son consecutivos y enlazados. Proporciona acceso por índice y métodos para buscar, ordenar y manipular datos.

```
List<Persona> personas = new List<Persona>(); // Creamos la lista vacía
```

```
var alba = new Persona("Alba", 22);
```

```
personas.Add(alba); // Añadimos una persona al final de la lista
```

```
personas.Add(new Persona("Ana", 23)); // Añadimos otra persona
```

```
var ana = personas[1];
```

```
Debug.Log(personas.Count); // 2
```

```
personas.Remove(alba); // Elimina a Alba de la lista
```

```
Debug.Log(personas.Count); // 1
```

```
personas.RemoveAt(0); // Elimina el primer elemento (Ana)
```

```
Debug.Log(personas.Count); // 0
```

```
foreach (var persona in personas)
```

```
    Debug.Log($"Nombre: {persona.nombre}, edad: {persona.edad}");
```

# C# Estructuras de datos: Dictionarios

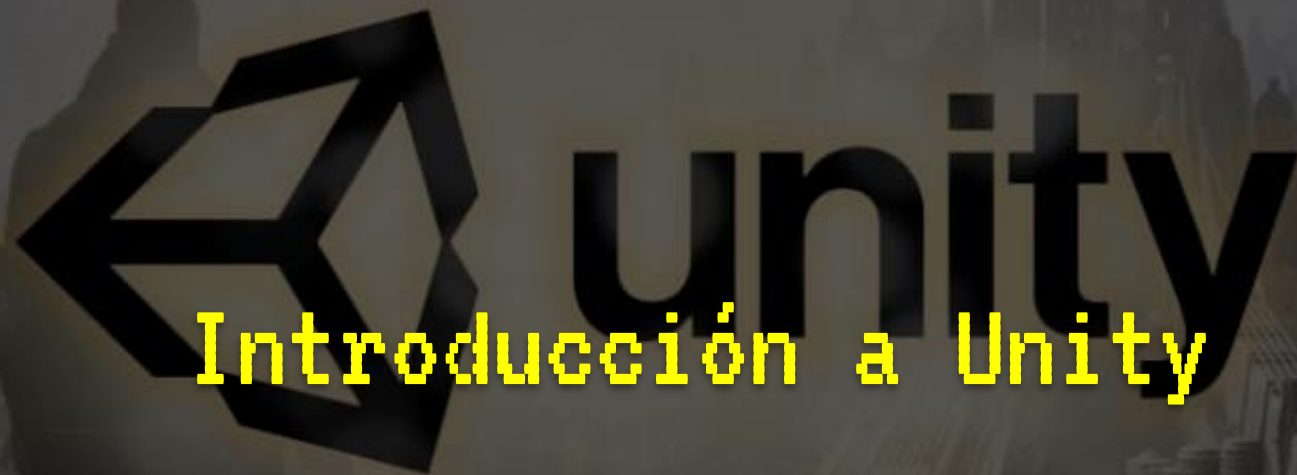
- Consiste en una colección de pares *clave-valor* ([key]-[value]) (System.Collections.Generic). Proporciona acceso a los elementos por clave, las cuales no pueden repetirse.

```
// Las claves serán de tipo string y los valores int
Dictionary<string, int> numeros = new Dictionary<string, int>();

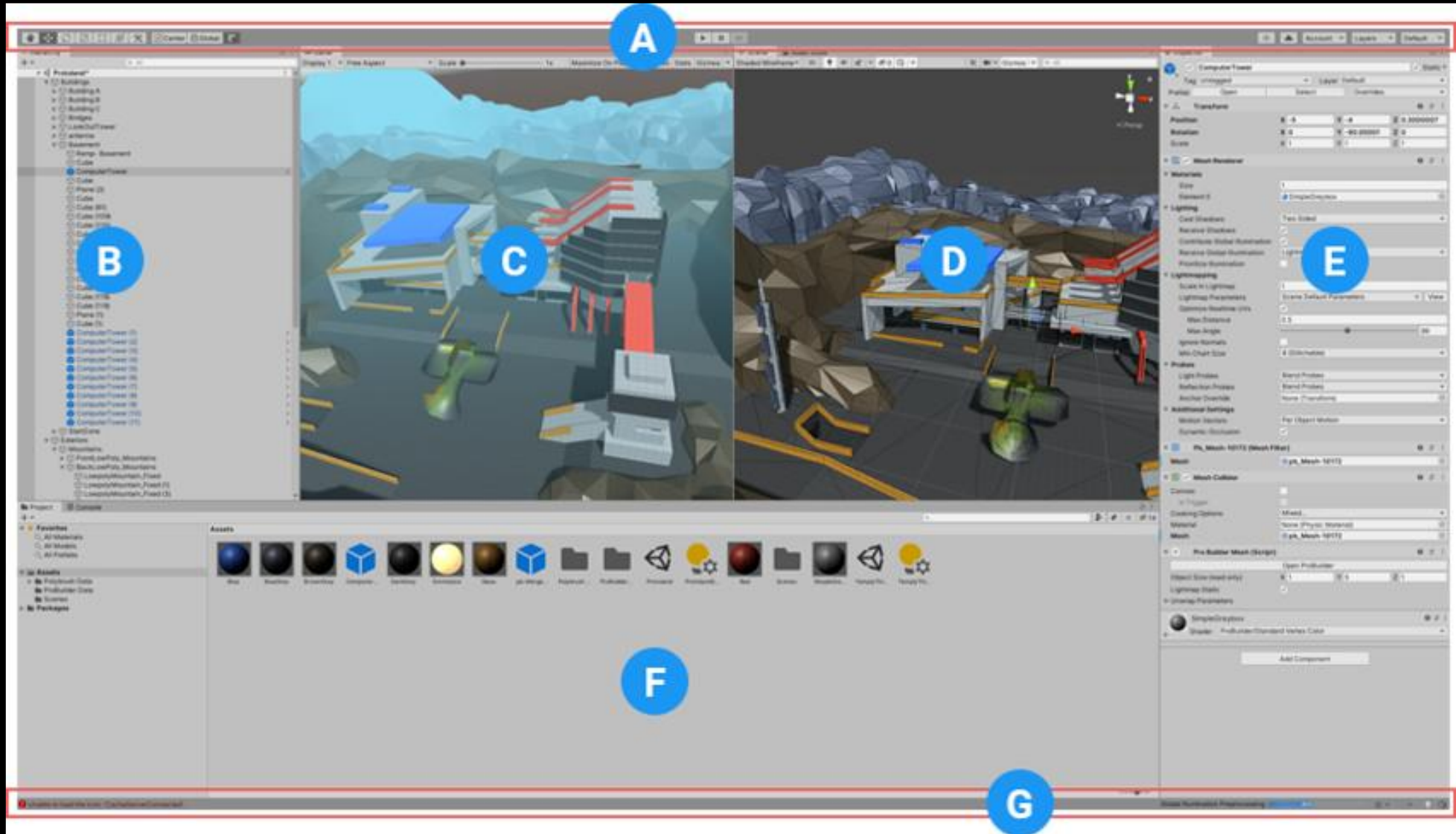
numeros["cero"] = 0;
numeros["siete"] = 7;
numeros.Add("cinco", 5);
numeros.Add("ocho", 8);

Debug.Log(numeros.Count); // 4
Debug.Log(numeros.Capacity); // 8

foreach (var valor in numeros.Values)
    Debug.Log(valor);
```



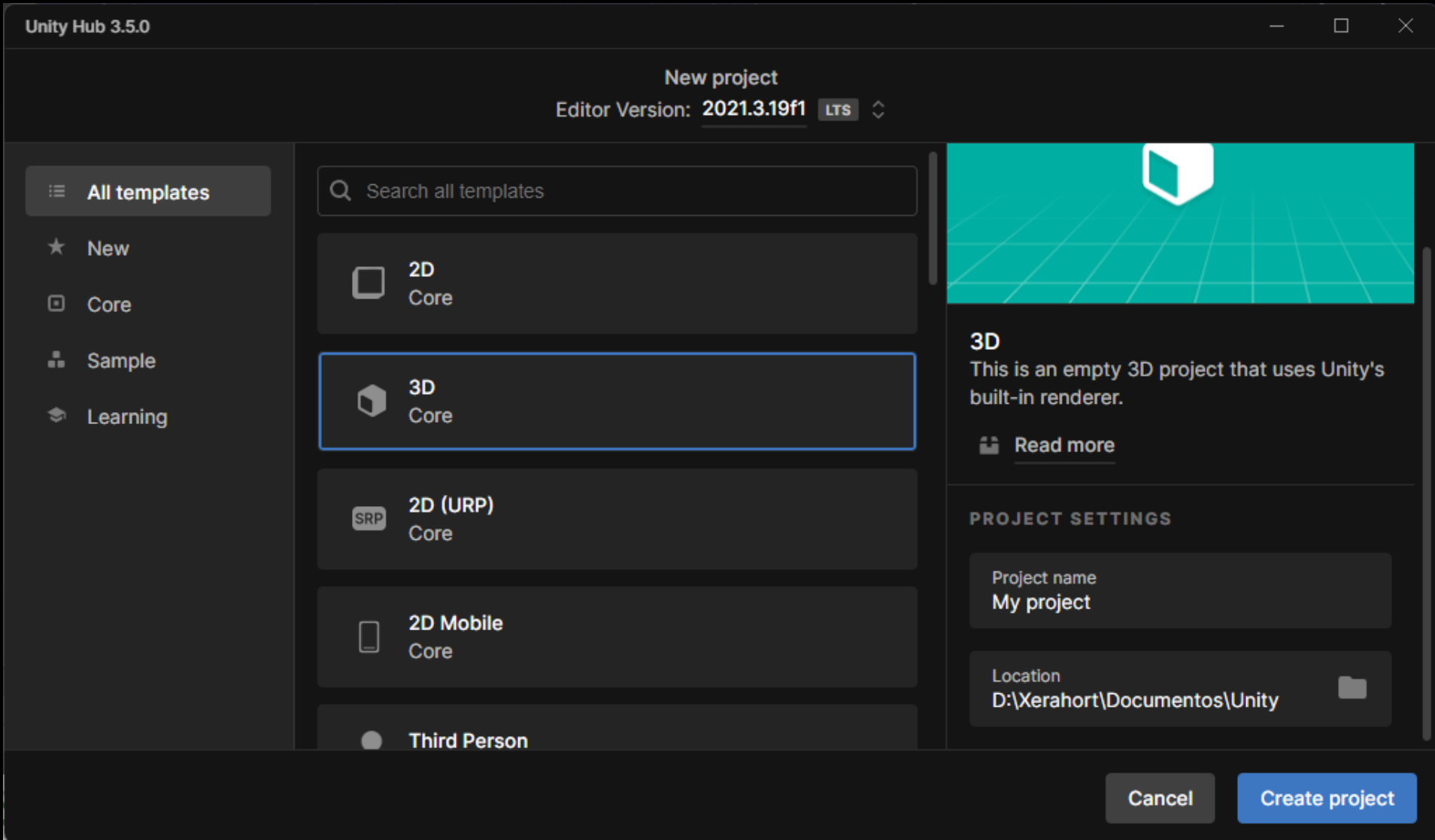
# Unity UI



# Unity UI

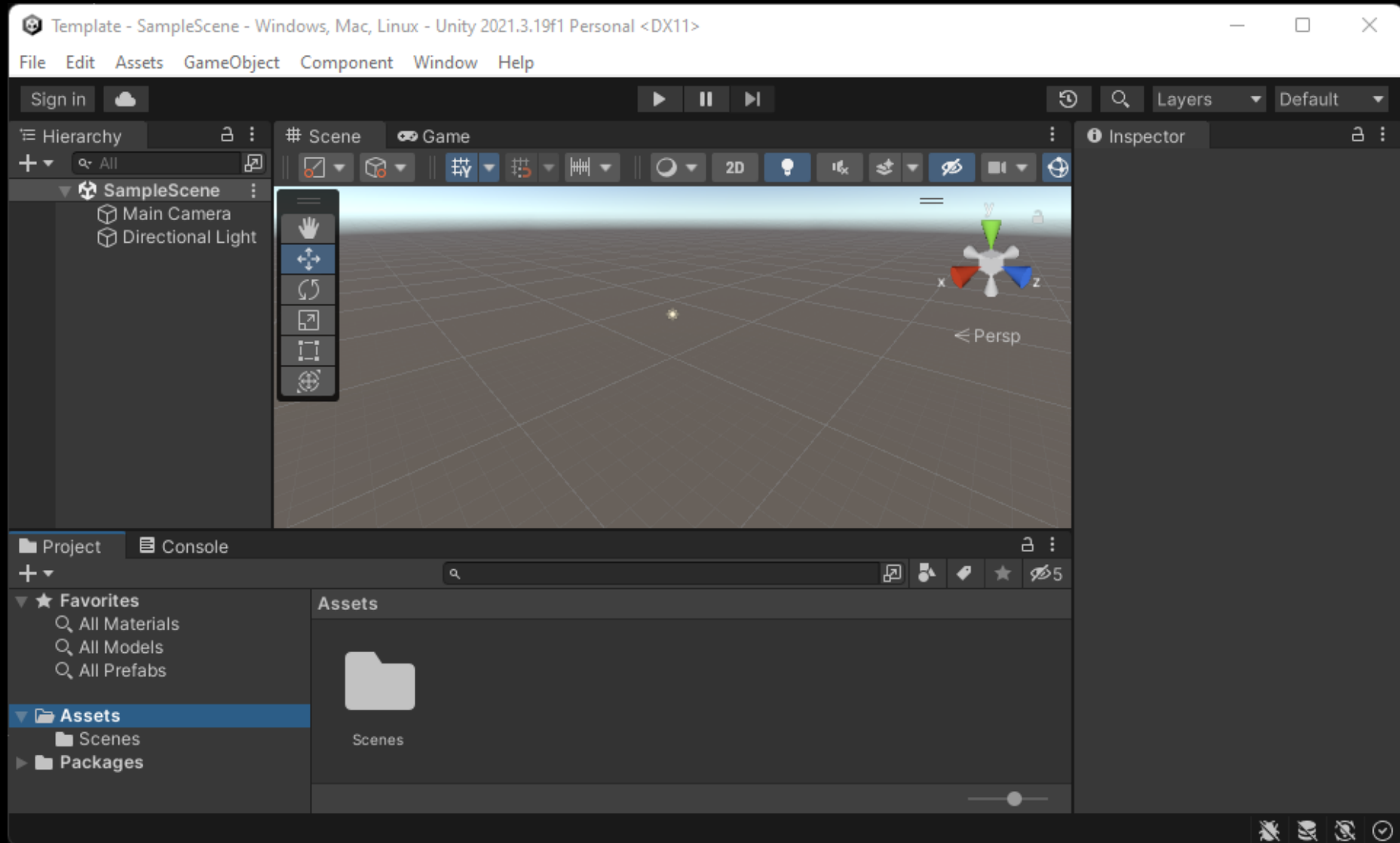
- A. La barra de Herramientas proporciona acceso a las funciones de trabajo más esenciales.
- B. La ventana de Jerarquía es una representación estructurada de cada *GameObject* en la Escena.
- C. La vista de Juego simula como se verá tu juego final renderizado a través de las Cámaras de tu Escena.
- D. La vista de Escena permite navegar sobre la escena y editarla.
- E. El Inspector permite ver y editar todas las propiedades del *GameObject* seleccionado.
- F. La ventana de Proyecto muestra la librería de *Assets* que están disponibles para usar en tu Proyecto.
- G. La barra de estado provee notificaciones acerca de varios procesos de Unity.

# Configuración proyecto





# Configuración proyecto



# Configuración proyecto

## Example 1

**.git**

Assets

+---Art

| +---Materials

| +---Models

| +---Textures

+---Audio

| +---Music

| \---Sound

+---Code

| +---Scripts # C# scripts

| \---Shaders # Shader files and shader graphs

+---Docs # Wiki, concept art, marketing material

+---Level # Anything related to game design in Unity

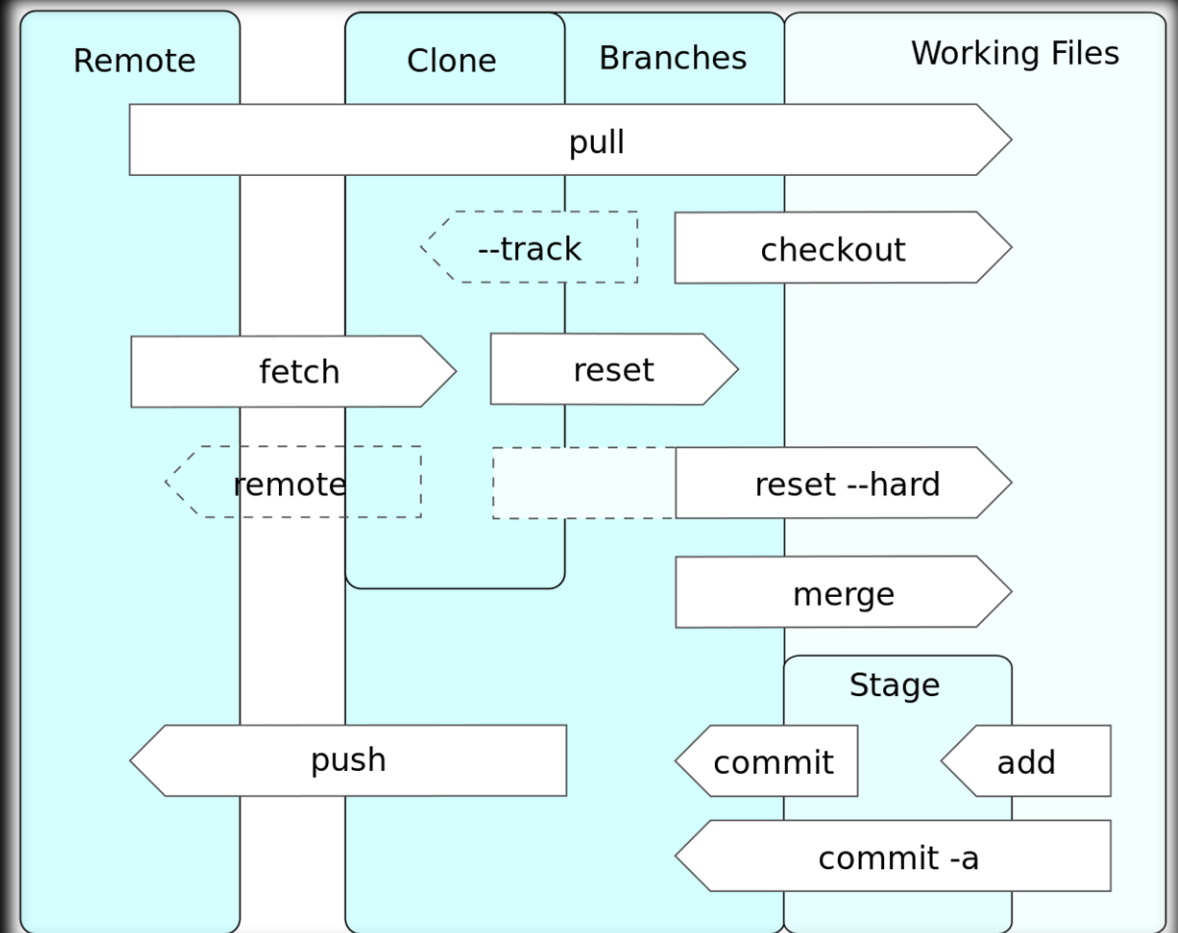
| +---Prefabs

| +---Scenes

| \---UI

# Control de versiones - Git

- Git es un sistema de control de versiones distribuido.
  - **Repositorio local** donde el desarrollador trabaja en su máquina.
  - **Repositorio remoto** donde se sincroniza su copia con la del servidor.
- Comandos
  - *Fork*: clonar repositorio remoto a otro.
  - *Clone*: clonar repositorio remoto en local.
  - *Checkout*: elegir rama del proyecto local.
  - *Pull*: actualizar rama local desde remoto.
  - *Add*: añadir ficheros nuevos a la pila (stage).
  - *Commit*: guardar cambios de la pila al repositorio local.
  - *Push*: mandar cambios (commits) de la rama al repositorio remoto.



# Control de versiones - Git

## Plataformas

- GitHub:  
<https://github.com>
- GitLab:  
<https://gitlab.com>

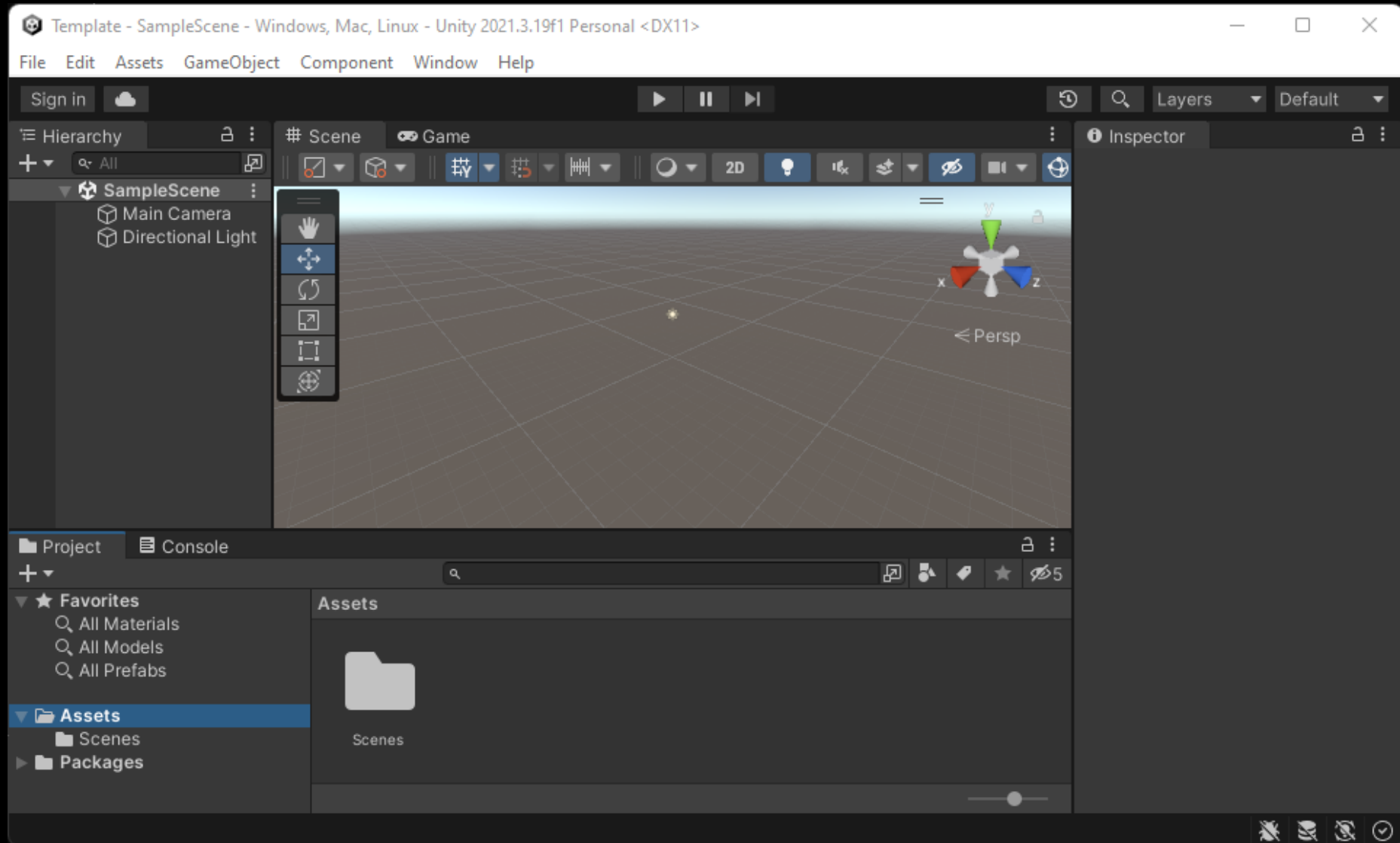
## Utilidades

- Control del código
- Integración continua
- Despliegue continuo
- Automatización QA
- Comunidad Open-Source

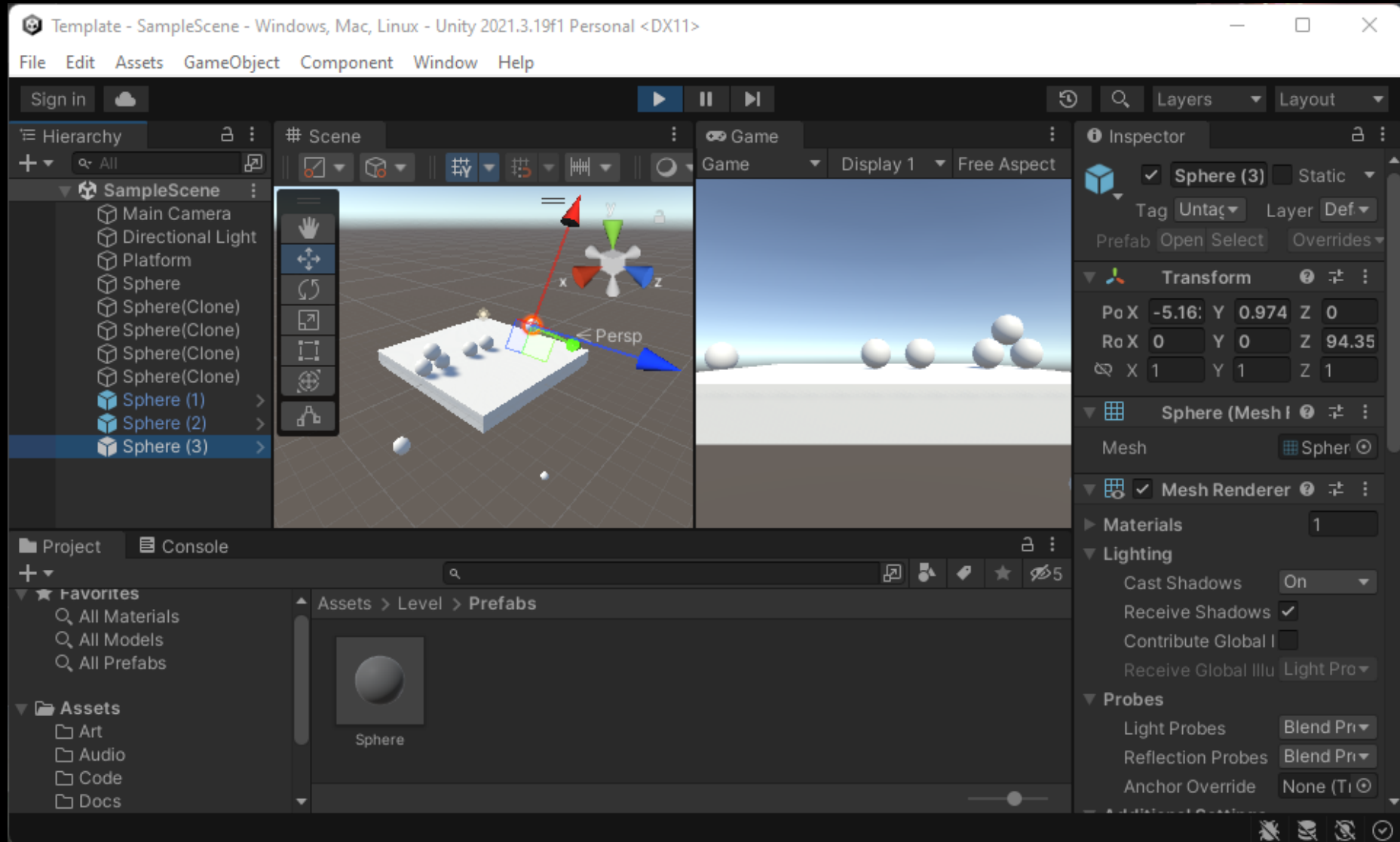
## Responsables

- En equipos departamentales:  
*DevOps*
- En equipos pequeños:  
*DevTeam*
- Recomendación para vuestro proyecto:
  - Coged práctica en vuestra primera iteración.
  - Al menos 1 miembro sea experto.
  - Plantilla disponible:  
<https://gitlab.com/upm-fdv/unity-project-template>

# Cámara y Escena



# Game Objects y Prefabs



Todo son prefabs :D





# Componentes

- Un **GameObject** está compuesto, como mínimo, de un componente **transform**, un nombre ("*GameObject*"), un tag ("*Untagged*"), y un layer ("*Default*").
- En la escena, siempre hay un **GameObject** con un componente **Camera** (*Main Camera*).
- Entender los componentes es entender Unity:

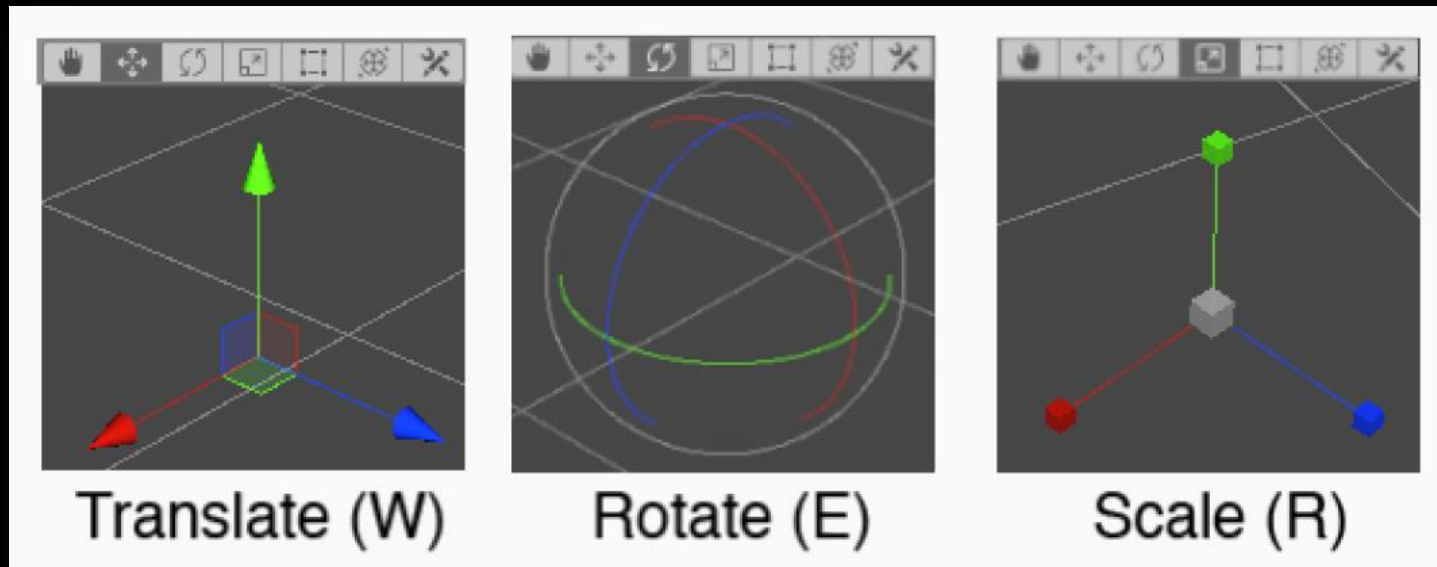
**Rigidbody, Collider, Particle System,  
Audio, Scripts,...**





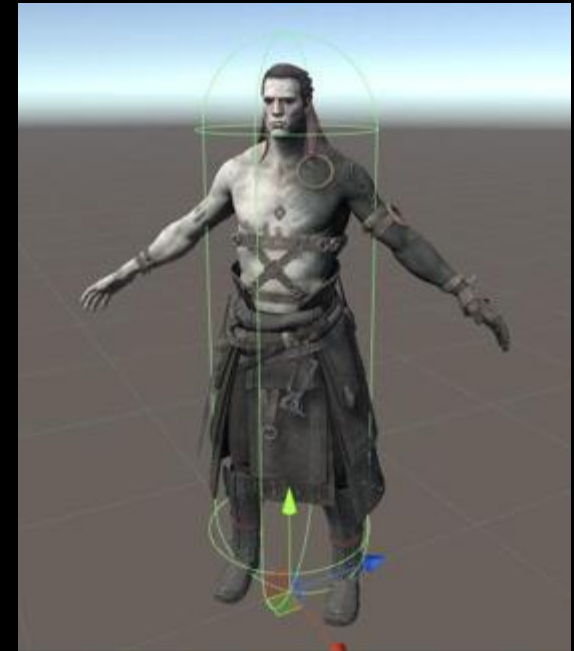
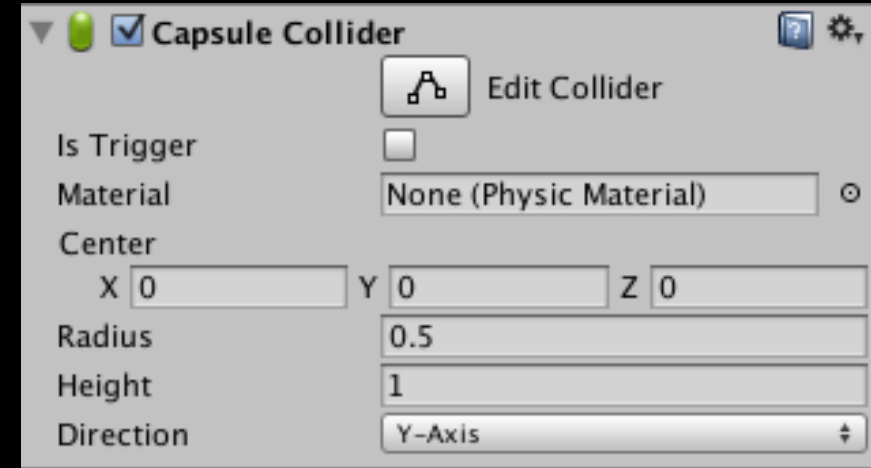
## Componentes: Transform

- El componente transform contiene la información sobre su posición, escala y rotación en la escena como una matriz
  - Translate: `Vector3(x,y,z)`.
  - Rotate: `Quaternion(x,y,z)` (`Quaternion.Identity`).
  - Scale: `Vector3(x,y,z)` (1 by default).



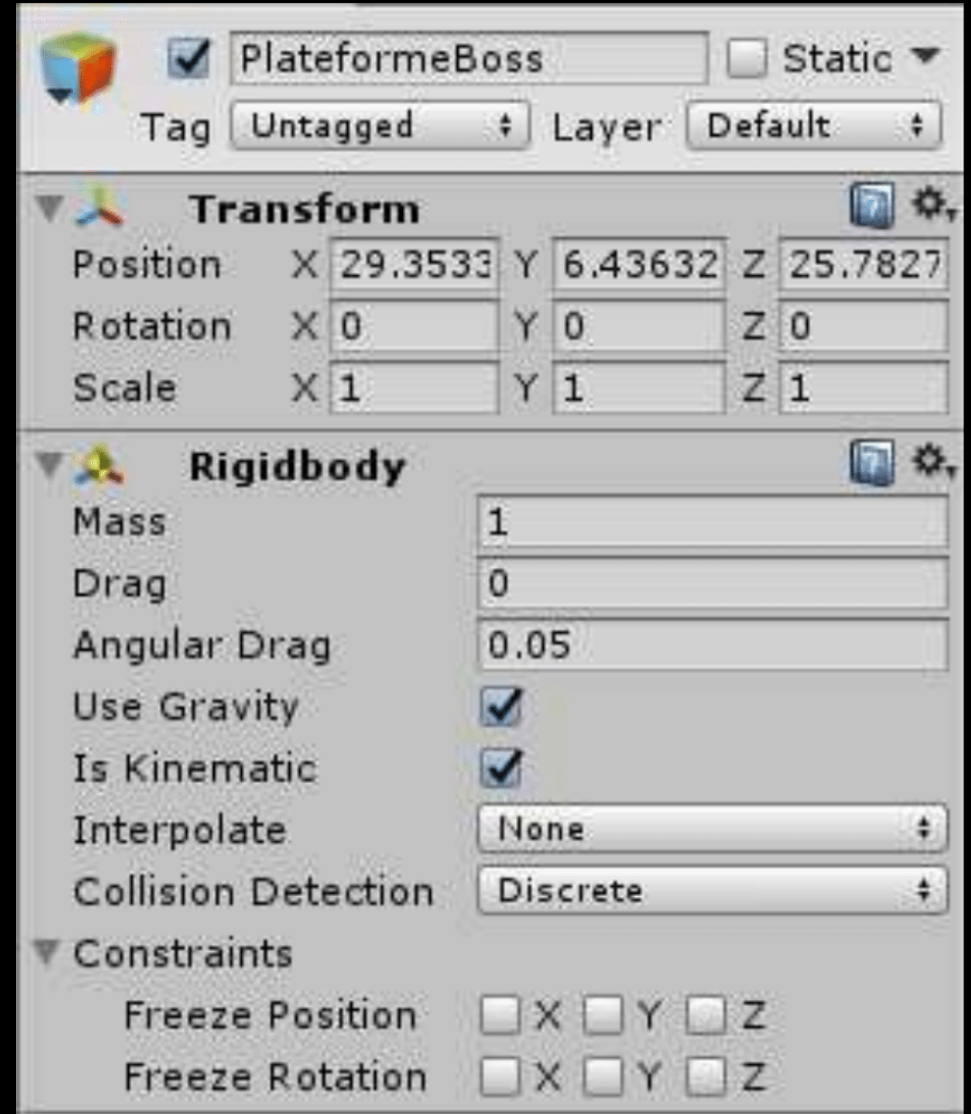
# Componentes: Colliders

- Los componentes Collider definen la forma de un objeto para los propósitos de colisiones:
  - Físicas
  - Atravesamientos ("Triggers")
- Un collider es invisible:
  - No tiene por qué tener la misma forma que el objeto ("mesh").
  - Una aproximación a menudo es más eficiente e indistinguible en el juego.
- Colliders compuestos
  - Combinar colisiones y triggers.
- Formas:
  - 3D: *Box*, *Sphere* y *Capsule Collider*.
  - 2D: *Box Collider 2D* y *Circle Collider 2D*.



# Componentes: Rigidbody

- **Rigidbody** es el componente principal que permite el comportamiento físico para un objeto.
  - El objeto adquiere masa y responde a la gravedad.
  - Versión 2D y 3D.
- El uso de Rigidbody asume el movimiento del objeto a través de **fuerzas**.
  - Si el GameObject tiene Colliders, el objeto puede ser movido por colisiones.
  - Cuidado con mezclarlo con el movimiento a través del componente Transform.



# Componentes: Script

- Los componentes Unity son clases que heredan de MonoBehaviour.
- Las variables públicas del componente son accesibles desde el inspector

```
1 using System.Collections;
2 using System.Collections.Generic;
3 using UnityEngine;
4
5 public class Persona : MonoBehaviour {
6     public string nombre;
7     public int edad;
8     private double genero;
9 }
```



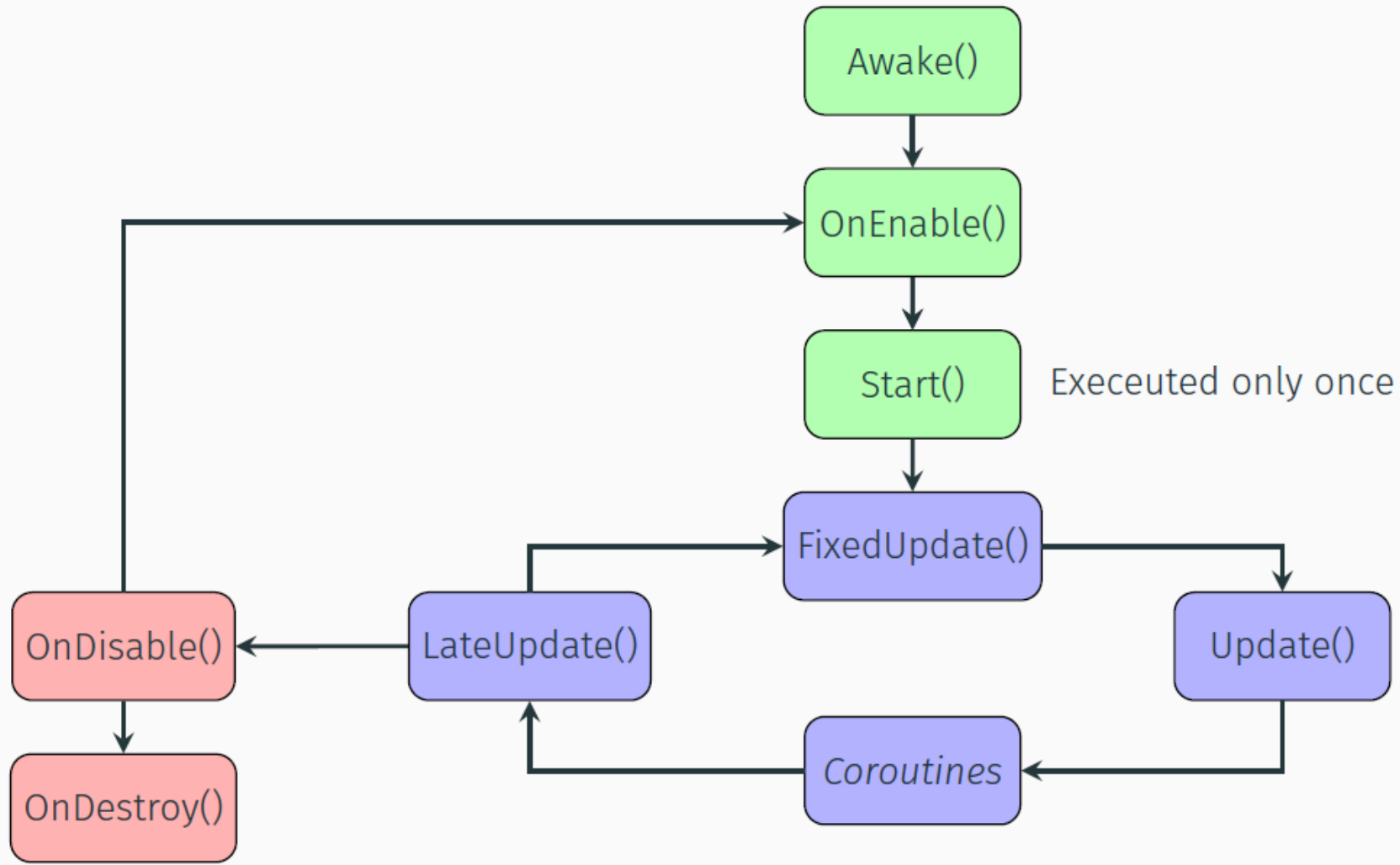
# C# Scripting: Esquema

```
1      using System.Collections;
2      using System.Collections.Generic;
3      using UnityEngine;
4
5      public class Player : MonoBehaviour
6      {
7          // Start is called before the first frame update
8          void Start()
9          {
10
11          }
12
13          // Update is called once per frame
14          void Update()
15          {
16
17          }
18      }
```

# MonoBehaviour Flowchart



# MonoBehaviour Flowchart





# MonoBehaviour Flowchart: Inicialización

- **Awake()**

Es la primera función de inicialización/creación que se ejecuta. Si el GameObject está deshabilitado cuando se crea, el Awake no se ejecutará hasta que se habilite.

- **OnEnable()**

Siempre que el GameObject esté activo, se ejecuta cuando el componente se activa o se instancia.

- **Start()**

Se ejecuta una sola vez antes del primer fotograma ("*frame*") activo del componente. Es decir, tanto el componente como el GameObject deben estar activos para que se ejecute.



# MonoBehaviour Flowchart: Físicas/Game Loop

- **Update()**

Se ejecuta una vez por frame. Su velocidad de repetición depende directamente de la ratio de fotogramas ("*framerate*").

- **FixedUpdate()**

Ocurre a un ritmo constante durante el juego ("*delta time*"), por lo que puede ser llamado más a menudo que *Update*.

- Puede ocurrir varias veces por fotograma si el tiempo entre fotogramas es muy alto (*framerate* bajo)...
- ...o no ocurrir entre dos frames si se ejecutan muy rápido (*framerate* alto).
- Las físicas se calculan justo después.

- **LateUpdate()**

Se ejecuta una vez por frame, una vez que han finalizado todas las funciones *Update* de todos los componentes de la escena en ese fotograma.

# MonoBehaviour Flowchart: Decommissioning

- **OnDisable()**

Se ejecuta cuando el componente o su GameObject es desactivado o destruido.

- **OnDestroy()**

Se ejecuta cuando finaliza la función Update (o LateUpdate, dependiendo de cuándo se produzca la destrucción del objeto) del último fotograma del componente.

# Scripting con GameObjects: Manipulación

```
1  using UnityEngine;
2
3  public class Controller : MonoBehaviour
4  {
5      //movement speed in units per second
6      private float movementSpeed = 5f;
7
8      void Update()
9      {
10         float horizontalInput = Input.GetAxis("Horizontal");
11         float verticalInput = Input.GetAxis("Vertical");
12
13         //update the position
14         transform.position = transform.position + new Vector3(
15             horizontalInput * movementSpeed * Time.deltaTime,
16             verticalInput * movementSpeed * Time.deltaTime,
17             0);
18
19         //output to log the position change
20         Debug.Log(transform.position);
21     }
22 }
```

# Scripting con GameObjects: Búsqueda

```
1  using UnityEngine;
2  using System.Collections;
3
4  public class Respawn : MonoBehaviour
5  {
6      public GameObject enemy;
7      public GameObject respawn;
8
9      void Start()
10     {
11         if (respawn == null)
12             respawn = GameObject.FindWithTag("Respawn");
13
14         enemy = GameObject.Find("Enemy");
15         State enemyState = enemy.GetComponent<State>();
16         enemyState.setHealth(100);
17
18         Instantiate(enemy, respawn.transform.position,
19                     respawn.transform.rotation);
20     }
21 }
```

# Scripting con GameObjects: Inputs

```
1  void Update() {  
2      if (Input.GetKeyDown(KeyCode.Space)) {  
3          Debug.Log("He pulsado espacio");  
4      }  
5      if (Input.GetKey(KeyCode.Space)) {  
6          Debug.Log("Estoy pulsando espacio");  
7      }  
8      if (Input.GetKeyUp(KeyCode.Space)) {  
9          Debug.Log("He dejado de pulsar espacio");  
10     }  
11 }
```

# Scripting con GameObjects: Instanciar, Destruir y Spawns

```
1  using System.Collections;
2  using System.Collections.Generic;
3  using UnityEngine;
4
5  public class SpamObject : MonoBehaviour {
6      public GameObject myPrefab;
7
8      void Update() {
9          if (Input.GetButtonDown("Fire1")) {
10              myPrefab.AddComponent<Persona>();
11              var clone = Instantiate(myPrefab,
12                                     new Vector3(0, 0, 0), Quaternion.identity);
13              Destroy (clone, 3.0f);
14          }
15      }
16  }
```

# Referencias

- Unity con Visual Studio Code <https://code.visualstudio.com/docs/other/unity>
- Unity con Visual Studio Community <https://visualstudio.microsoft.com/es/vs/unity-tools/>
- Unity con JetBrains Rider <https://www.jetbrains.com/idea/dotnet-unity>
- Documentacion de C# <https://learn.microsoft.com/es-es/dotnet/csharp/>
- Microsoft Learn C# <https://learn.microsoft.com/es-es/users/dotnet/collections/yz26f8y64n7k07>
- Recursos Unity:
  - <https://docs.unity3d.com/es/2021.1/Manual/UnityOverview.html>
  - <https://docs.unity3d.com/es/2021.1/Manual/UsingTheEditor.html>
  - <https://docs.unity3d.com/es/2021.1/Manual/PhysicsOverview.html>
  - <https://docs.unity3d.com/Manual/ExecutionOrder.html>
  - <https://learn.unity.com/tutorial/input-manager>
- Recursos JS:
  - <https://unity.com/how-to/organizing-your-project>
  - <https://learngitbranching.js.org>
  - <https://www.w3schools.com/git/>
  - <https://gitlab.com/upm-fdv/unity-project-template>

Curso 2025-26

# L3-1: Unity3D Introducción y Scripting (fin)

Fundamentos de Videojuegos

Jose María Barambones  
DLSIS - ETSI Informáticos  
Universidad Politécnica de Madrid